🐳 Docker Architecture & Docker Daemon – Creative Theory Notes

1. Overview of Docker Architecture

Docker follows a **client–server architecture**, where different components work together to build, run, and manage containers efficiently.

At a high level, Docker architecture consists of:

- **Docker Client**
- **Docker Daemon**
- **Docker Registry (Remote Repositories)**

These components communicate with each other to perform all Docker operations.

---

2. Docker Client

The **Docker Client** is the primary interface through which users interact with Docker.

Key points:

- Installed as part of the Docker setup
- Accepts commands from users via the **CLI** or Docker Desktop
- Examples of commands: pull, run, build, stop, remove

The Docker client does **not execute commands directly**. Instead, it sends requests to the Docker daemon for processing.

---

3. Docker Daemon (dockerd)

The **Docker Daemon**, commonly referred to as `dockerd`, is a background service that performs all core Docker operations.

Responsibilities of the Docker daemon:

- Builds Docker images
- Pulls images from registries
- Creates, runs, stops, and removes containers
- Manages Docker networks
- Manages Docker volumes

The daemon continuously listens for requests from the Docker client and processes them accordingly.

---

4. Communication via Docker API

The Docker client and Docker daemon communicate using a **REST API**.

Workflow:

1. User enters a Docker command
2. Docker client sends an API request
3. Docker daemon receives and processes the request
4. Docker daemon returns the response to the client

This API-based communication allows Docker to work both locally and remotely.

---

5. Docker Registries

A **Docker Registry** is a storage system for Docker images.

Types of registries:

- **Public registry** – e.g., Docker Hub
- **Private registry** – Used by organizations for internal images

Docker registries allow:

- Pulling images for use
- Pushing custom images
- Sharing images across systems

The Docker daemon interacts directly with registries to download or upload images.

---

6. Image Flow in Docker Architecture

When an image-related command is issued:

- Docker client sends request to daemon
- Docker daemon checks if the image exists locally
- If not found:
    - Docker daemon pulls the image from a remote registry
- Image is stored locally and used to create containers

This automated flow simplifies image management.

---

7. Container Management in Architecture

Containers are managed entirely by the Docker daemon.

The daemon:

- Creates containers from images
- Allocates system resources
- Handles container lifecycle (start, stop, pause, remove)
- Maintains container isolation

The client simply acts as a command sender and status receiver.

---

8. Summary of Docker Architecture Flow

The Docker architecture works in a clear and structured manner:

- **User → Docker Client → Docker Daemon → Registry / System Resources**
- Client sends commands
- Daemon executes them
- Registries provide images

This separation of responsibilities improves security, scalability, and maintainability.

---

9. Transition to Application Requirements (Java Example)

After explaining Docker architecture, the discussion transitions to application requirements using **Java** as an example.

Key insight:

- To develop Java applications, a **JDK (Java Development Kit)** is required
- JDK includes:
    - Compiler
    - Debugging tools
    - Profiling tools

A **JVM alone is insufficient** for development, as it only runs compiled Java code.

---

10. Relevance to Docker

Docker allows developers to:

- Package the **JDK and application together**
- Ensure consistent Java environments across machines
- Avoid installing development tools directly on host systems

This highlights Docker’s role in simplifying application development and deployment.

---

11. Conclusion

Docker's client–daemon architecture ensures a clean separation between command execution and user interaction. The **Docker daemon (`dockerd`)** acts as the engine behind all Docker operations, while registries serve as centralized image storage.

Understanding this architecture is essential before moving on to:

- Docker images and builds
- Application containerization
- Advanced Docker workflows
